Aufgabenblatt 3

Ziel des Blattes

Nachdem die Rollen verteilt sind und alle nach der Recherche mit den grundlegenden Anforderungen des Projekts vertraut sind, kann es nun eigentlich losgehen. Um ein größeres Projekt erfolgreich und termintreu zu bewältigen ist es sinnvoll, noch einige Energie auf die korrekte Aufstellung des Teams zu verwenden, so dass neben der eigentlichen Software-Entwicklung auch die begleitenden Maßnahmen – insbesondere das Qualitätsmanagement und die Dokumentation – im Blickfeld bleiben.

Neben dem **Abschluss der Projektspezifikation** in Lastenheft und Glossar, welche Gegenstand des ersten Reviews sein werden, soll das Team sich intern über die **Grundanforderungen an das Projektmanagement** verständigen und diese in entsprechenden Dokumenten verbindlich fixieren. Als erste Fingerübung ist weiterhin eine kleine **Softwarestudie** zu erstellen, in der die zu verwendenden Technologien schon einmal exemplarisch eingesetzt werden. Dazu gibt es ein Zusatzblatt, in dem dieser Teil der Aufgabenstellung themenspezifisch konkretisiert ist.

Softwarequalität und Dokumentation

Die **Qualität einer Software** wird neben ihren funktionalen Leistungsparametern auch wesentlich durch die Qualität des Codes bestimmt. Ein Maß für diese Qualität ergibt sich aus den verschiedenen Geschäftsanforderungen, die in unterschiedlichen Phasen des Softwarelebenszyklus entstehen. Dabei stehen Fragen des Bugfixings, der Wartbarkeit und Änderbarkeit im Vordergrund, für welche es wichtig ist, dass sich auch andere, projektfremde Entwickelnde schnell in Designaspekte der Software einarbeiten können.

Die Einhaltung einer Reihe von **allgemeinen Prinzipien** unterstützt diese Anforderungen:

- der Quellcode ist gut strukturiert, lesbar und folgt den "Regeln für guten Code",
- Quellcode und Design sind gut dokumentiert, sowohl intern als auch extern,
- zu den einzelnen Softwareeinheiten existiert Testmaterial, das alle wesentlichen Funktionen und Zweige abdeckt.

Es macht Sinn, sich an bereits aufgestellten Coding Conventions zu halten (z.B. für Java zu finden unter http://www.oracle.com/technetwork/java/index-135089.html oder http://www.torsten-horn.de/techdocs/java-codingconventions.htm).

Es ist sowohl ein Aspekt der Selbstkontrolle als auch Aufgabe des Softwarequalitätsmanagements, die Einhaltung dieser Regeln zu überwachen. Im Team sind Verantwortliche für Dokumentation und Test festgelegt. Deren Aufgabe besteht darin, die erforderlichen Qualitätsstandards festzulegen und deren Einhaltung zu organisieren und zu kontrollieren. Dazu ist ein Dokumentationskonzept und (in einem späteren Arbeitsblatt) ein Testkonzept zu vereinbaren, schriftlich zu fixieren und später zu einem **Qualitätssicherungsplan** weiterzuentwickeln.

Die **Dokumentation** des Designs und des Quellcodes soll sich aus zwei wesentlichen Komponenten zusammensetzen: 1. der internen, quelltextnahen Dokumentation, die z.B. bei Java mit **Javadoc** (siehe http://www.oracle.com/technetwork/java/javase/documentation/writingdoccomments-137785.html) extrahiert werden kann, sowie einer **speziellen Design-Dokumentation**, in der alle wichtigen Designaspekte verständlich dargelegt und begründet sind.

Ein zu erstellendes **Testkonzept** soll für die Komponententests einzelner Softwareeinheiten die **Werkzeuge eines Testframeworks** (bei Java wäre das JUnit, siehe http://junit.org/) einsetzen.

Ziel von Komponententestfällen ist es, einen Indikator dafür zu haben, ob das System bzw. einzelne Komponenten so funktionieren wie sie sollen. Dazu dient eine Vielzahl von Testbeispielen, die alle mehr oder weniger wichtigen funktionalen Aspekte und Programmzweige abdecken sollen und (in der

Regel) *vor* dem eigentlichen Programmieren der Applikation oder der Änderungen erstellt werden. Die Beschreibung der Testfälle ist Bestandteil des zu erstellenden Testkonzepts.

Diese Testfälle müssen nach jeder Änderung am System durchgearbeitet (d.h. getestet) werden.

Es wird überdies stark dazu geraten, dass das Team Tests automatisieren soll. Hierzu bietet sich z.B. Jenkins (http://jenkins-ci.org/) oder CircleCI (https://circleci.com/) an. Hierbei handelt es sich um Werkzeuge zur kontinuierlichen Integration. Auch Versionskontroll-Werkzeuge (so auch Github und Gitlab) bieten Werkzeuge für die Automatisierung.

Für die Kommunikation und Qualitätssicherung im Team macht es ebenso Sinn, Werkzeuge wie z.B. Redmine (http://www.redmine.org/) einzusetzen. Falls Sie kein großes Ticket-System benötigen, greifen Sie auf den Issue Tracker von Github oder Gitlab zurück. Eine weitere Alternative wären die Projekttools Jira (https://www.atlassian.com/de/software/jira) oder Asana (https://asana.com/).

Die Installation dieser Werkzeuge allein bringt allerdings sehr wenig. Vielmehr sind *Absprachen* über den Gebrauch im Team zu treffen. Diese sind zu dokumentieren und selbstverständlich regelmäßig auf Einhaltung zu prüfen!

Aufgaben:

1. Mit den Regeln für guten Java-Code (und HTML/ Javascript/ etc.) vertraut zu machen und im Projekt zu befolgen.
2. Mit dem Dokumentationskonzept der gewählten Programmiersprache vertraut machen.
3. Mit dem Unit-Framework für Komponententests vertraut machen.
4. Den Einsatz von Werkzeugen zur Unterstützung der Softwareerstellungs- und Qualitätssicherungsprozessen überlegen und die Entscheidungen bzw. Festlegungen hierzu dokumentieren.

Abzugeben: Dokumentationskonzept, in dem Qualitätsstandards, Vorgehensweisen und Verantwortlichkeiten für die verschiedenen Teile der Dokumentation fixiert sind.

Es gibt keine weitergehenden allgemeingültigen Ansätze, um Softwarequalität unabhängig von Einsatzgebiet und Aufgabenstellung zu sichern. Allerdings existieren für einzelne Anwendungsbereiche Erfahrungen, welche Designprinzipien und Architekturkonzepte im jeweiligen Gebiet besonders erfolgreich waren und deshalb weit verbreitet sind. Neue Projekte ziehen doppelten Nutzen, wenn sie solchen Konzepten folgen: die Konzepte bürgen einerseits bereits für eine gewisse Qualität der Produkte und sind andererseits auch den Entwicklenden vertraut, die später mit dem Code zu tun haben werden. Zu denken ist im Rahmen dieses Projektes beispielsweise an die Benutzung von **Design Patterns**.

Einarbeitung in das Fachkonzept

Zur Einarbeitung in das Fachkonzept ist eine kleine Softwarestudie anzufertigen, in der einige grundlegende Aspekte des Themas schon einmal prototypisch realisiert werden sollen. Die Aufgabenstellung ist themenspezifisch in einem Zusatzblatt weiter untersetzt.

Bei der Erstellung der Softwarestudie ist arbeitsteilig vorzugehen:

- Zunächst ist ein neues Projekt anzulegen, welches gleich entsprechende Versionierung vorsieht (Einsatz von Git).
- Das Team überlegt gemeinsam, wie das Design des Prototyps aussehen soll und verteilt die Implementierungsarbeiten.
- Das vereinbarte Dokumentationskonzept kann bereits in dieser Aufgabenstellung angewendet werden.

- Eine kleine Designbeschreibung des Prototyps ist anzulegen. *(Testmaterial sowie Testautomation oder sogar Continous Integration wird für diese Studie nicht gefordert.)*

Die **Designbeschreibung** enthält alle wichtigen Informationen zu den Struktur- und Entwurfsprinzipien Ihrer Software, welche eine durchschnittliche, bisher mit dem Projekt nicht befasste Programmiererfahrene Person, kennen sollte, bevor sie sich in Ihre Software einarbeitet, um Änderungen oder Ergänzungen des Quellcodes vorzunehmen (und dabei die detailliertere, etwa Javadoc-basierte Dokumentation des Quellcodes zu nutzen).

In der Designbeschreibung sind also alle wichtigen Struktur- und Entwurfsentscheidungen zu begründen.Insbesondere ist zu erläutern, mit welchen Struktur- und Entwurfsentscheidungen welche Konzepte und Ansätze umgesetzt wurden.
Folgende Hauptpunkte enthält die Designbeschreibung:

1. Allgemeines
2. Produktübersicht
    - Beschreibung der äußerlichen Funktionsmerkmale Ihrer Anwendung
3. Grundsätzliche Struktur- und Entwurfsentscheidungen für Ihre Anwendung
    - Was sollte eine bisher unbeteiligte SW-entwickelnde Person über das Gesamtsystem gelesen haben, ehe sie sich Gliederungspunkt 4. und der Javadoc-Dokumentation zuwendet? Insbes.: Wie sieht die Gesamtarchitektur der Anwendung aus?
4. Grundsätzliche Struktur- und Entwurfsentscheidungen der einzelnen Pakete/Komponente der Anwendung
    - Was sollte eine Person gelesen haben, nachdem sie Gliederungspunkt 3. kennt und ehe sie sich der Javadoc-Dokumentation eines speziellen Pakets zuwendet?

Aufgabe:

Obige Designbeschreibung erstellen und das Design implementieren, d.h. codieren der Anwendung.

Abzugeben (bzw. für die Betreuenden zugänglich zu machen) sind:

- Die Designbeschreibung der Softwarestudie
- Code -Dateien entsprechend der Aufgabenstellung
- Präsentation der Lösung auf den Webseiten der Gruppe. Gemeint ist hier eine irgendwie funktionierende Demo

Abschluss der Anforderungsanalyse

Aufgabe:

Auf Basis der Recherche in Aufgabenblatt 2 sind die Rahmen für die eigene Projektarbeit abzustecken und in Lastenheft und Glossar festzuhalten. Dabei ist an die Gliederungsforderungen in [1] anzuknüpfen. Das Lastenheft soll mehr Funktionalität beschreiben als im Rahmen des Projekts implementiert werden kann. Die Muss- und Kann-Ziele sowie der Abschnitt „Produktfunktionen“ des Lastenhefts sind weiter in mehrere Etappen zu gliedern, die gegebenenfalls in Nachfolgeprojekten umgesetzt werden können.

Abzugeben: Glossar und Lastenheft

Review des Lastenhefts und der Designstudie

Zu Glossar und Lastenheft wird das **erste Review** stattfinden, in welchem der Umfang des Projekts für Ihre Gruppe vereinbart wird. Auf dieser Basis ist dann das Pflichtenheft auszuarbeiten.

Ebenso ist die Designstudie (inkl. Vorführung des Programmes) Gegenstand des Reviews.

Im Review wird die Anforderungsanalyse anhand der vorgelegten Dokumente und der Demo-Präsentation durchgesprochen.

Damit sich alle Seiten auf die Reviews gut vorbereiten können, ist die **rechtzeitige Vorlage** von Lastenheft und Glossar **Voraussetzung**. Zu beachten ist zudem das **Merkblatt zum Ablauf eines Reviews**.

Literaturverzeichnis

1. Balzert, Helmut; Liggesmeyer, Peter (2011): Lehrbuch der Softwaretechnik: Entwurf, Implementierung, Installation und Betrieb. 3. Aufl. Heidelberg: Spektrum Akademischer Verlag (Lehrbücher der Informatik).